

Hobby OS Kernel Roadmap

Limine + your own kernel, using MIT 6.828 (2018) labs 2–6 as a conceptual reference — skip 6.828 Lab 1, Limine replaces it.

How to use this: day numbers are a **relative sequence**, not a calendar — assume roughly 1–3 focused hours per “day.” Go slower or faster as needed; the order of topics matters more than the pace. “Study” = read/understand before coding. “Code” = what to actually build/commit that day. You are not doing the JOS labs literally (JOS is 32-bit and has its own build system) — you’re building the equivalent functionality yourself, in your own 64-bit Limine-based kernel, using the JOS lab write-ups as a conceptual spec/reference and OSDev Wiki for Limine-specific mechanics.

Core references used throughout: OSDev Wiki (osdev.org) — “Limine Bare Bones,” “Meaty Skeleton,” “GDT,” “IDT,” “Paging,” “Higher Half Kernel” pages; Limine Bootloader Protocol spec (github.com/limine-bootloader/limine); Limine barebones template repo; and MIT 6.828 2018 lab pages (pdos.csail.mit.edu/6.828/2018/labs/) for lab2–lab6 as design references. Intel/AMD manuals for anything hardware-specific.

Phase 0 — Toolchain, Limine Hello World, GDT/IDT (replaces 6.828 Lab 1)

Goal: a kernel that boots via Limine, prints to screen/serial, and won't silently triple-fault when something goes wrong.

Day	Study	Resources	Code
Day 1	Cross-compiler concept; why you can't use your host gcc for a freestanding kernel.	OSDev: “GCC Cross-Compiler”	Build an x86_64-elf cross toolchain (binutils + gcc, freestanding, no red zone).
Day 2	Limine boot protocol: requests/responses, higher-half loading, memory map, framebuffer.	Limine protocol spec (LIMINE.md); Limine barebones repo	Clone limine-bootloader/limine-barebones as a starting skeleton; get it building and booting in QEMU unmodified.
Day 3	Linker scripts; ELF sections; why the kernel links at a high virtual address.	OSDev: “Linker Scripts”, “Higher Half Kernel”	Read and annotate the barebones linker.ld line by line until you can explain every line.
Day 4	Serial (UART) output basics; framebuffer text alternative.	OSDev: “Serial Ports”, “COM Ports”	Write kprintf()/serial_write() so you can print strings and hex numbers from kernel code.
Day 5	x86-64 GDT refresher (segments still required even in long mode); why Limine's GDT is fine to reuse short-term.	OSDev: “Global Descriptor Table”	Print the GDT Limine set up (sanity check) — no need to rewrite it yet.
Day 6–7	IDT structure, interrupt gates vs trap gates, exception vs interrupt, error codes.	OSDev: “Interrupt Descriptor Table”, “Exceptions”	Implement idt_init() with handlers for vectors 0–31 (divide error, GPF #13, page fault #14 especially) that print register state and halt instead of triple-faulting.
Day 8	Recap + buffer.	—	Trigger a deliberate null-pointer dereference and confirm you get a readable page-fault message, not a reboot.

Phase 1 — Physical & Virtual Memory Management (≈ 6.828 Lab 2)

Goal: track free physical pages, build/extend page tables yourself, understand physmap and higher-half addressing (this is exactly the territory Mark's stream was in).

Day	Study	Resources	Code
Day 9	Physical vs virtual vs linear addresses; Limine's memory map response format.	6.828 Lab 2 intro + "Virtual, Linear, and Physical Addresses" section; Limine protocol — memmap request	Parse Limine's memory map request response; print each region's base/length/type.
Day 10–11	Bump allocator concept (this is literally what offlinemark built).	6.828 Lab 2 boot_alloc() section	Implement a simple bump/boot-time allocator over usable memory regions.
Day 12	Physmap pattern: linearly mapping all physical RAM into high virtual address space.	OSDev: "Higher Half Kernel", "Identity Paging"; Limine HHDM (higher-half direct map) request	Use Limine's HHDM request to get your physmap base; write phys_to_virt()/virt_to_phys() helpers.
Day 13–15	Free-list physical page allocator design; reference counting concept.	6.828 Lab 2 — page_init/page_alloc/page_free, PageInfo struct	Implement a frame allocator: struct Page[] array or bitmap, page_alloc()/page_free(), refcounts.
Day 16	x86-64 4-level paging structures (PML4/PDPT/PD/PT), page table entry flags.	OSDev: "Paging"; Intel SDM Vol 3A Ch 4	Write pte flag constants and helper macros for indexing PML4/PDPT/PD/PT from a virtual address.
Day 17–19	Page table walk/create; mapping/unmapping a page; recursive vs physmap-based table access.	6.828 Lab 2 — pgdir_walk(), boot_map_region(), page_insert()/page_remove()	Implement your own vmm_map()/vmm_unmap()/vmm_walk() using the physmap to reach table pages.
Day 20–21	Kernel address space layout planning: where kernel image, physmap, heap, MMIO go.	OSDev: "Memory Map (x86)"; 6.828 Lab 2 Part 3	Write out and document your own virtual memory map (like inc/memlayout.h) as a header file/comment block.
Day 22	The 2GB-image problem Mark hit; why large static objects should move off the image.	(the stream summary you gave me earlier)	Move any large global buffers into boot-time bump-allocated memory + placement-new/manual construction, exactly like Mark did.
Day 23	Recap, fix bugs found via #PF handler from Phase 0.	—	Stress test: map/unmap randomly, allocate until OOM, confirm no leaks/corruption via assertions.

Phase 2 — Kernel Heap, ELF Loading, User Mode, Syscalls (≈ 6.828 Lab 3)

Goal: run untrusted user-mode code, handle its exceptions/syscalls, isolate it from the kernel.

Day	Study	Resources	Code
Day 24–25	Kernel heap allocator design (now that page-granularity alloc exists).	OSDev: “Memory Allocation” (simple bump/free-list/slab overview)	Implement kmalloc()/kfree() on top of your page allocator (simple free-list or slab is fine to start).
Day 26–27	ELF64 format: headers, program headers, loading segments into memory.	OSDev: “ELF”; 6.828 Lab 3 — load_icode()	Write an ELF64 loader that maps PT_LOAD segments into a fresh address space.
Day 28	Per-task address space (page directory per process/env) and switching CR3.	6.828 Lab 3 — env_setup_vm()	Add a process/task struct with its own top-level page table; implement address-space switch.
Day 29–30	GDT additions for ring 3 (user code/data segments), TSS and kernel stack switch on privilege change.	OSDev: “GDT Tutorial”, “Task State Segment”	Extend your GDT with user segments + TSS; set RSP0 so interrupts from user mode land on a kernel stack.
Day 31–32	Entering ring 3 for the first time (iretq); IDT entries for syscall path.	6.828 Lab 3 — env_pop_tf() / “An Example” section; OSDev: “Getting to Ring 3”	Build a minimal user-mode ELF (‘hello world’ that just loops or triggers int) and jump into it via iretq.
Day 33–34	Syscall mechanism: int 0x80/int 0x30 style vs SYSCALL/SYSRET instruction; argument passing convention.	6.828 Lab 3 — System calls section; Intel SDM SYSCALL/SYSRET	Implement a syscall entry point and a small syscall table (start with just ‘write’/‘exit’).
Day 35–36	Validating user-supplied pointers before the kernel dereferences them (critical security boundary).	6.828 Lab 3 — user_mem_check / page faults and memory protection section	Implement a pointer-range validator used by every syscall that takes a user pointer.
Day 37	Recap.	—	Run 2–3 tiny user programs end-to-end: print via syscall, cause an intentional fault, confirm clean process termination not kernel panic.

Phase 3 — Multiple Processes, Scheduling, fork(), IPC (≈ 6.828 Lab 4)

Goal: run more than one process, switch between them, let them create each other and talk to each other.

Day	Study	Resources	Code
Day 38–39	Process/task table design; process states (runnable/running/blocked/dead).	6.828 Lab 4 Part A intro; struct Env in inc/env.h (as a design reference)	Define your task struct + task table/free list.
Day 40–42	Context switching: saving/restoring registers, kernel stacks per task.	OSDev: “Context Switching”; 6.828 Lab 4 — per-CPU kernel stacks	Implement context_switch(old, new) in asm + C wrapper; test with 2 cooperative tasks.
Day 43–44	Timer interrupt (PIT or APIC timer) for preemption.	OSDev: “Programmable Interval Timer” or “APIC Timer”	Wire up a periodic timer interrupt; on each tick, call your scheduler.
Day 45–46	Round-robin scheduling.	6.828 Lab 4 — sched_yield() section	Implement round-robin scheduler; run 3+ tasks that print and yield/preempt.
Day 47–49	fork() semantics; copying vs sharing an address space; copy-on-write concept.	6.828 Lab 4 Part B — User-level page fault handling & COW fork	Implement fork(): duplicate address space (start with full-copy ‘dumbfork’ style; COW is a stretch goal).
Day 50–51	Inter-process communication design (message passing vs shared memory).	6.828 Lab 4 Part C — IPC section	Implement a minimal IPC syscall pair (send/recv) between two tasks.
Day 52–54	Locking basics for kernel data shared across interrupts/tasks (even single-core: interrupts re-entering the kernel).	OSDev: “Locks”; 6.828 Lab 4 — big kernel lock discussion	Add spinlocks around your scheduler/allocator/task-table; disable interrupts during critical sections as needed.
Day 55	Recap + stress test.	—	Run a fork-tree style test: parent forks several children, children exit, parent reaps them, no crashes/leaks.

Phase 4 — Storage, File System, Spawn, Shell (≈ 6.828 Lab 5)

Goal: persist data to disk, load programs from a filesystem instead of embedding them in the kernel image, get an interactive shell.

Day	Study	Resources	Code
Day 56–57	Disk access basics on real/emulated hardware: AHCI (SATA) or virtio-blk (much easier under QEMU) vs legacy IDE.	OSDev: “AHCI” or “VirtIO” (recommend virtio-blk for a hobby kernel under QEMU)	Implement a minimal block driver: read_sector()/write_sector().
Day 58–59	Filesystem design tradeoffs; on-disk layout (superblock, inodes/files, data blocks).	6.828 Lab 5 — On-Disk File System Structure section	Design your own simple on-disk FS format (or start even simpler: a read-only ‘initrd’/tar-like image first).
Day 60–62	Block cache / buffer cache design; demand-loading blocks via page faults (optional advanced technique) vs simple in-memory cache.	6.828 Lab 5 — Block Cache section	Implement a simple block cache (hash map block# -> in-memory buffer) with a flush/dirty mechanism.
Day 63–65	Free space tracking (bitmap allocator for disk blocks); path resolution & directory walking.	6.828 Lab 5 — Block Bitmap & File Operations sections	Implement alloc_block()/free_block(), plus open/read/write/create for files and directories.
Day 66–67	Loading and running a program from disk (‘spawn’) instead of an embedded binary.	6.828 Lab 5 — Spawning Processes section	Wire your ELF loader (Phase 2) to read the binary from your filesystem instead of a linked-in blob.
Day 68–69	Keyboard driver + console input; file descriptor abstraction (stdin/stdout).	OSDev: “PS/2 Keyboard”; 6.828 Lab 5 — keyboard interface & file descriptor sections	Implement keyboard IRQ handler + a simple fd table (console read/write, at minimum).
Day 70–73	Shell design: parse a command line, spawn a process, wait for it, pipe/redirect basics.	6.828 Lab 5 — The Shell section	Write a tiny user-space shell: run a program by name, support at least one of pipes or redirection.
Day 74–75	Recap.	—	Milestone: boot to your shell, list files, run a couple of user programs from disk interactively.

Phase 5 — Networking (≈ 6.828 Lab 6, optional “final project” tier)

6.828 itself calls this lab the “default final project” — open-ended, least hand-held. Treat this phase as optional / stretch; many hobby kernels stop before this.

Day	Study	Resources	Code
Day 76–78	PCI bus enumeration; how a NIC is discovered and its BARs negotiated.	OSDev: “PCI”; 6.828 Lab 6 — PCI Interface section	Implement PCI bus scanning; detect and print your virtual NIC (e1000 or virtio-net under QEMU).
Day 79–82	NIC driver model: MMIO registers, DMA descriptor rings (TX/RX).	OSDev: “VirtIO” (simpler) or Intel 8254x (e1000) manual via 6.828 Lab 6 links	Implement TX: transmit a hand-crafted raw Ethernet frame and confirm via QEMU pcap/tcpdump.
Day 83–85	Receive path; interrupt vs polling for incoming packets.	6.828 Lab 6 — Receiving Packets section	Implement RX ring + receive syscall/queue; receive and print an incoming frame.
Day 86–88	Minimal network stack: Ethernet -> ARP -> IPv4 -> ICMP/UDP (or bring in a tiny existing TCP/IP stack like lwIP/picoTCP instead of writing your own).	OSDev: “ARP”, “IPv4”, “ICMP”; lwIP docs if you choose to integrate it	Get your kernel to answer ARP requests and reply to ping (ICMP echo) from the host.
Day 89–90	Sockets-ish API for user space; simple echo/HTTP server as a capstone.	6.828 Lab 6 — Network Server & Web Server sections	Build a tiny echo server or static-file HTTP server running as a user process on your kernel.

That’s the full arc from “Limine hands off control” to “a kernel with processes, a filesystem, a shell, and basic networking” — everything 6.828 labs 2–6 cover, built your own way. Total is roughly 90 focused sessions; realistically, spread over several months alongside other commitments (this is close to what a ‘week 127’ streamer’s pace looks like — slow and steady is normal for this kind of project).